ASSIGNMENT- 12.5

Name: Rishitha Kunchala

HT.No: 2303A51333

Batch: 20

Task 1: Merge Sort Implementation.

Implement Merge Sort using AI assistance. Include time and space complexity in the docstring and verify with test cases.

Prompt: # Generate a Python function merge_sort(arr) that sorts a list in ascending order. Include time complexity and space complexity in the docstring.

Code:

```python
#Task 1 – Merge Sort Implementation
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)


def merge(left, right):
    sorted_list = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            sorted_list.append(left[i])
            i += 1
        else:
            sorted_list.append(right[j])
            j += 1

    sorted_list.extend(left[i:])
    sorted_list.extend(right[j:])

    return sorted_list


arr = [38, 27, 43, 3, 9, 82, 10]
```

Result:

```
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
```

Observation:

Merge Sort guarantees O(n log n) time complexity in all cases. It is stable and suitable for large datasets. However, it requires extra space for merging.

Task 2: Binary Search

Implement binary search for sorted lists with complexity explanation.

Prompt: # Generate a Python function

binary_search(arr, target) that returns index or

Include best, average, worst-case complexities in

docstring.

Code :

```python
#Task 2 – Binary Search
def binary_search(arr, target):
    """
    Performs Binary Search on sorted list.

    Best Case: O(1)
    Average Case: O(log n)
    Worst Case: O(log n)
    Space Complexity: O(1)
    """
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return -1


arr = [1, 3, 5, 7, 9, 11]
print(binary_search(arr, 7))
```

Result:

```
3
```

Observation:
Binary Search reduces search space by half each iteration, making it highly efficient for sorted datasets.

Task 3: Healthcare Appointment Scheduling
Search appointments by ID and sort by consultation fee.

Prompt: # Recommend and implement efficient search and sorting for appointment records..

Code:

```python
#Task 3 – Healthcare Appointment Scheduling
appointments = [
    {"id": 101, "patient": "A", "fee": 500},
    {"id": 102, "patient": "B", "fee": 300},
    {"id": 103, "patient": "C", "fee": 700}
]
```

Result:

```
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
3
Sorted by Fee: [{'id': 102, 'patient': 'B', 'fee': 300}, {'id': 101, 'patient': 'A', 'fee': 500}, {'id': 103, 'patient': 'C', 'fee':
```

Observation:
Sorting by fee helps prioritize billing. Efficient searching ensures quick appointment retrieval.

Task 4: Railway Reservation System
Search tickets by ID and sort by seat number.

Prompt: # Implement search and sorting for railway booking system.

Code:

Assign12.5.py > ...

```python
#Task 4 – Railway Reservation System
tickets = [
    {"ticket_id": 1, "seat": 12},
    {"ticket_id": 2, "seat": 5},
    {"ticket_id": 3, "seat": 20}
]

sorted_tickets = sorted(tickets, key=lambda x: x["seat"])
print(sorted_tickets)
```

Result:

```
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
3
Sorted by Fee: [{'id': 102, 'patient': 'B', 'fee': 300}, {'id': 101, 'patient': 'A', 'fee': 500}, {'id': 103, 'patient': 'C', 'fee': 700}]
[{'ticket_id': 2, 'seat': 5}, {'ticket_id': 1, 'seat': 12}, {'ticket_id': 3, 'seat': 20}]
```

Observation
Sorting by seat helps structured allocation. Efficient searching speeds up ticket retrieval

Task 5: Hostel Room Allocation.

Search by student ID and sort by room number.

Prompt: # Suggest and implement optimized search and sort for hostel allocation system.

Code:

```
Assign12.5.py > ...
#Task 5 – Hostel Room Allocation
rooms = [
    {"student_id": 101, "room": 12},
    {"student_id": 102, "room": 5},
    {"student_id": 103, "room": 20}
]

sorted_rooms = sorted(rooms, key=lambda x: x["room"])
print(sorted_rooms)
```

Result:

```
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
3
Sorted by Fee: [{'id': 102, 'patient': 'B', 'fee': 300}, {'id': 101, 'patient': 'A', 'fee': 500}, {'id': 103, 'patient': 'C', 'fee': 700}]
[{'ticket_id': 2, 'seat': 5}, {'ticket_id': 1, 'seat': 12}, {'ticket_id': 3, 'seat': 20}]
[{'student_id': 102, 'room': 5}, {'student_id': 101, 'room': 12}, {'student_id': 103, 'room': 20}]
```

Observation:
Sorting organizes room allocation efficiently. Binary search enables fast record lookup.

Task 6: Online Movie Streaming Platform

Search by movie ID and sort by rating.

Prompt: # Implement searching and sorting for movie platform.

Code:

```
Assign12.5.py > ...
#Task 6 – Online Movie Streaming Platform
movies = [
    {"id": 1, "rating": 8.5},
    {"id": 2, "rating": 9.2},
    {"id": 3, "rating": 7.8}
]
```

Result:

```
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
3
Sorted by Fee: [{'id': 102, 'patient': 'B', 'fee': 300}, {'id': 101, 'patient': 'A', 'fee': 500}, {'id': 103, 'patient': 'C', 'fee': 700}]
[{'ticket_id': 2, 'seat': 5}, {'ticket_id': 1, 'seat': 12}, {'ticket_id': 3, 'seat': 20}]
[{'student_id': 102, 'room': 5}, {'student_id': 101, 'room': 12}, {'student_id': 103, 'room': 20}]
[{'id': 2, 'rating': 9.2}, {'id': 1, 'rating': 8.5}, {'id': 3, 'rating': 7.8}]
```

Observation:
Sorting by rating improves recommendation systems. Efficient search helps retrieve movies quickly.

Task 7: Task 7 – Agriculture Crop Monitoring

Search crop data by ID and sort by moisture level.

Prompt: # Implement search and sort for

agriculture monitoring system.

Code:

```
#Task 7 – Agriculture Crop Monitoring
crops = [
    {"id": 1, "moisture": 30},
    {"id": 2, "moisture": 45},
    {"id": 3, "moisture": 25}
]
```

Result:

```
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
3
Sorted by Fee: [{'id': 102, 'patient': 'B', 'fee': 300}, {'id': 101, 'patient': 'A', 'fee': 500}, {'id': 103, 'patient': 'C', 'fee': 700}]
[{'ticket_id': 2, 'seat': 5}, {'ticket_id': 1, 'seat': 12}, {'ticket_id': 3, 'seat': 20}]
[{'student_id': 102, 'room': 5}, {'student_id': 101, 'room': 12}, {'student_id': 103, 'room': 20}]
[{'id': 2, 'rating': 9.2}, {'id': 1, 'rating': 8.5}, {'id': 3, 'rating': 7.8}]
[{'id': 3, 'moisture': 25}, {'id': 1, 'moisture': 30}, {'id': 2, 'moisture': 45}]
```

Observation:
Sorting by moisture helps prioritize irrigation. Efficient search improves crop monitoring.

Task 8: Task 8 – Airport Flight Management

Search flight details by ID and sort by departure time.

Prompt: # Implement searching and sorting for airport flight system.

Code:

```
#Task 8 – Airport Flight Management
flights = [
    {"id": 1, "departure": "10:30"},
    {"id": 2, "departure": "08:15"},
    {"id": 3, "departure": "12:45"}
]

sorted_flights = sorted(flights, key=lambda x: x["departure"])
print(sorted_flights)
```

Result:

```
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
3
Sorted by Fee: [{'id': 102, 'patient': 'B', 'fee': 300}, {'id': 101, 'patient': 'A', 'fee': 500}, {'id': 103, 'patient': 'C', 'fee': 700}]
[{'ticket_id': 2, 'seat': 5}, {'ticket_id': 1, 'seat': 12}, {'ticket_id': 3, 'seat': 20}]
[{'student_id': 102, 'room': 5}, {'student_id': 101, 'room': 12}, {'student_id': 103, 'room': 20}]
[{'id': 2, 'rating': 9.2}, {'id': 1, 'rating': 8.5}, {'id': 3, 'rating': 7.8}]
[{'id': 3, 'moisture': 25}, {'id': 1, 'moisture': 30}, {'id': 2, 'moisture': 45}]
[{'id': 2, 'departure': '08:15'}, {'id': 1, 'departure': '10:30'}, {'id': 3, 'departure': '12:45'}]
```

Observation:
Sorting by departure ensures correct scheduling order. Efficient search helps retrieve flight details instantly